

Top 20 Useful HTML Tags & Features -Part-1

1 <form> – Data sender

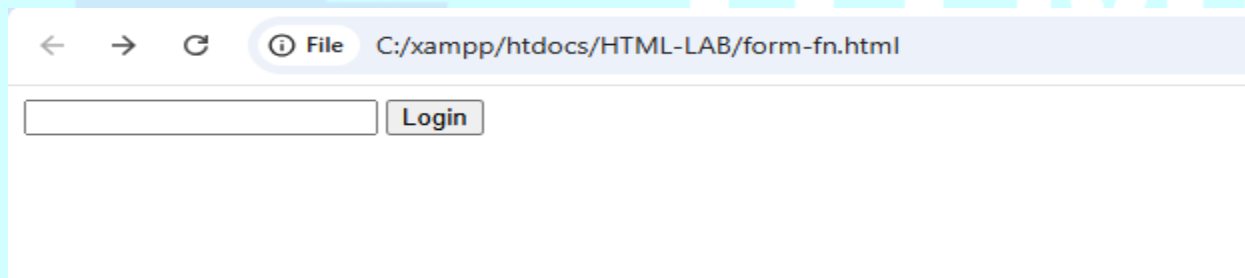
<form> tags to collect and submit user data to a server

Syntax

```
<form action="/login" method="POST">
  <input name="user">
  <button>Login</button>
</form>
```

Output

- ⇒ Login form appears
- ⇒ Data sent to server



🔒 **Security:** Input goes to backend → SQLi, brute-force target

Example:

1. User types "**Ram**" in the text field
2. Clicks the "**Login**" button
3. Browser sends this **POST** request to the server:

```
POST /login HTTP/1.1
Content-Type: application/x-www-form-urlencoded

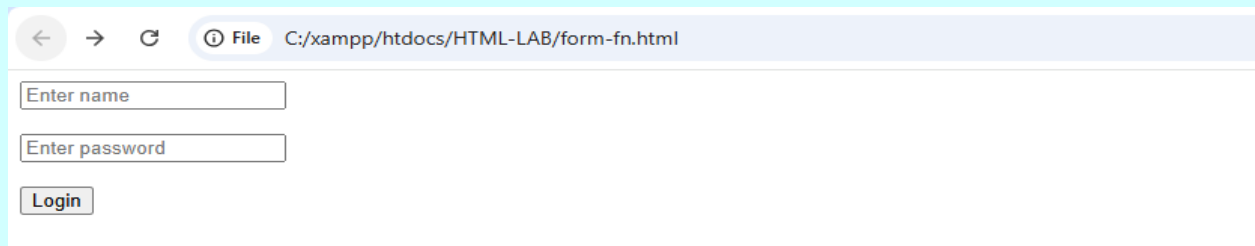
user=Ram
```

4. Server receives this data at the **/login** endpoint

2 <input type="text">

<input> is an HTML element that collects user data like text or passwords and sends it to the server for processing.

```
<form>
  <input type="text" name="user" placeholder="Enter name"><br><br>
  <input type="password" name="password" placeholder="Enter
password"><br><br>
  <button>Login</button>
</form>
```



A screenshot of a web browser window. The address bar shows the file path 'C:/xampp/htdocs/HTML-LAB/form-fn.html'. The browser displays a simple login form with two text input fields. The first field has the placeholder text 'Enter name' and the second field has the placeholder text 'Enter password'. Below the input fields is a button labeled 'Login'.

1. User Interaction

- User types "Ram" in first field
- User types "secret123" in password field
- User clicks "Login" button

2. Browser Process

1. Form Validation: Browser checks required fields
2. Data Encoding: Converts form data to URL format
3. Request Creation: Prepares HTTP POST request

3. HTTP Request Sent

```
POST /login HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 29
```

```
user=Ram&password=secret123
```

4. Server Processing

- Receives at /login endpoint
- Parses user=Ram and password=secret123
- Validates credentials in database
- Responds with success/failure page

5. Browser Response

- Receives server response (HTML page)
- Renders new page to user
- Redirects to dashboard or shows error

Complete Flow: User Input → Browser Encoding → HTTP Request → Server Processing → Response → Browser Display

HTML

```
<input type="password">
```

```
<input type="password" name="password">
```

Output

➡ Dots instead of characters

🔒 **Security: Masking** ≠ encryption

<button>

```
<button type="submit">Submit/Login</button>
```

Output

➡ Click sends request

🔒 **Security:** Triggers HTTP request

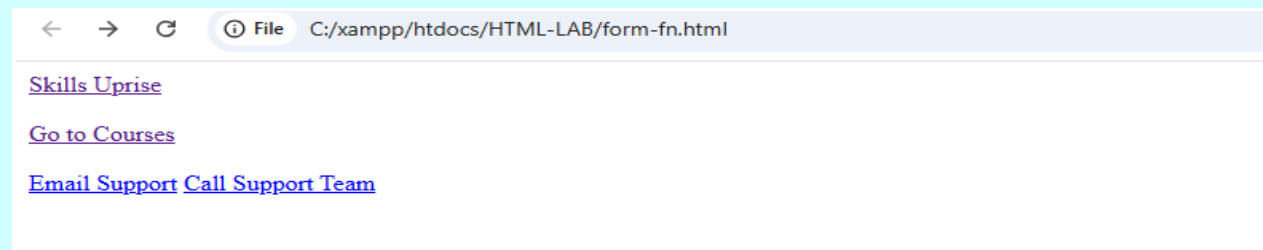
3 <a href>

<HTML TAGS>

<a href> creates clickable links in webpages. It connects one page to another or to files/emails.

Users click them to navigate to different locations.

```
a href="https://skillsuprise.com/">Skills Uprise</a><br><br>
<a href="https://skillsuprise.com/courses">Go to Courses</a>
<a href="mailto:support@example.com">Email Support</a>
<a href="tel:+111134567890">Call support Team</a>
```



4 <script>

<script> tag adds JavaScript to webpages to make them do things like popups, animations, and form checks.

```
<script>alert("It's a Trap")</script>
```

1. Shows Popup Alert: When this code runs, a browser popup appears with the message "It's a Trap"
2. Common XSS Attack: This is a basic example of a Cross-Site Scripting (XSS) payload used in security testing

